

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины «Искусственный интеллект и машинное обучение»**

Выполнила:

Федорова Дарья Юрьевна 2 курс,
группа ИВТ-б-о-24-1, 09.03.01
«Информатика и вычислительная
техника», направленность
(профиль) «Программное
обеспечение средств
вычислительной техники и
автоматизированных систем»,
очная форма обучения

(подпись)

Руководитель практики:
Воронкин Р. А.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

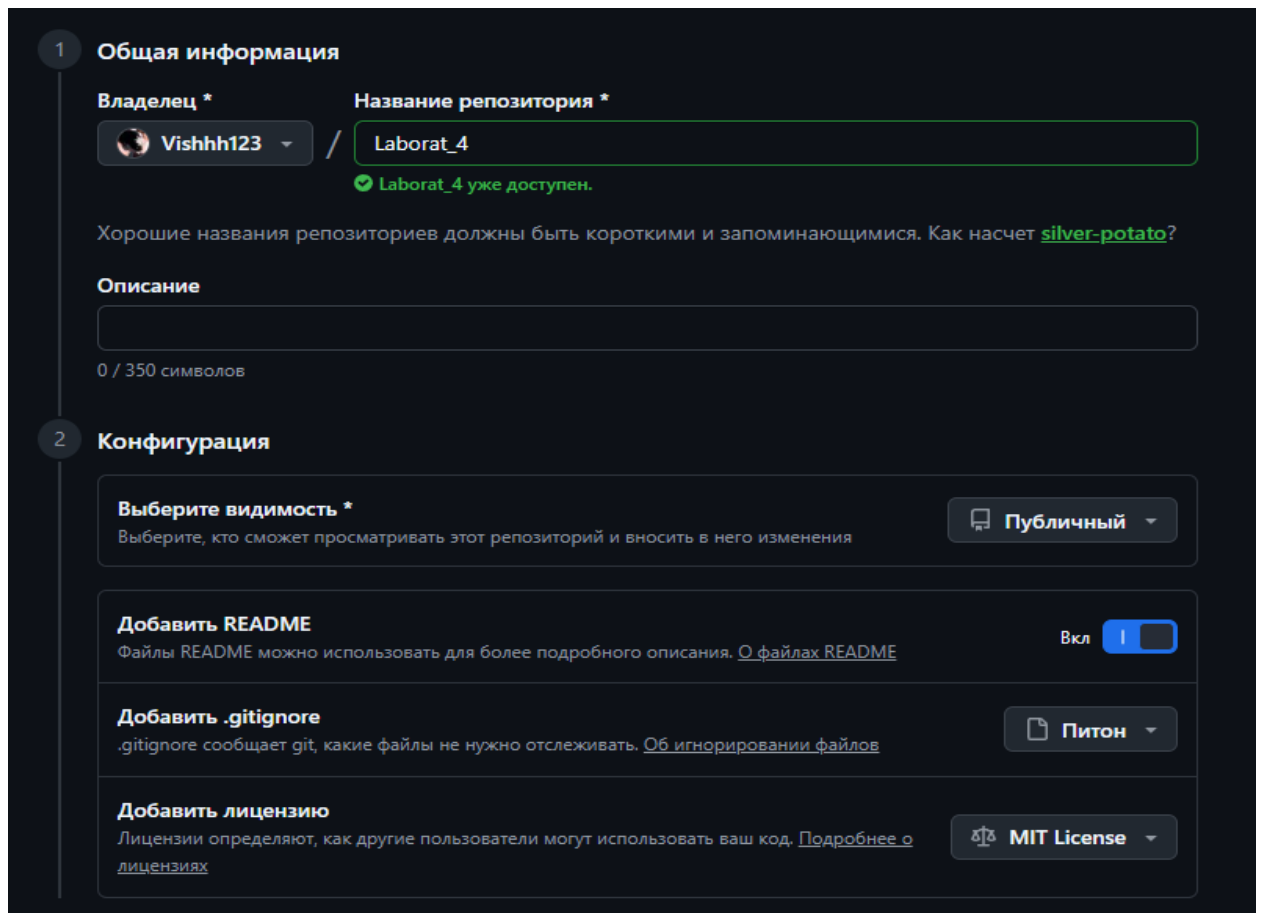
Вариант 25

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Ссылка на репозиторий: https://github.com/Vishhh123/Laborat_4

Ход работы:



1 **Общая информация**

Владелец * **Vishhh123** / Название репозитория * **Laborat_4**

✓ Laborat_4 уже доступен.

Хорошие названия репозитория должны быть короткими и запоминающимися. Как насчет [silver-potato](#)?

Описание

0 / 350 символов

2 **Конфигурация**

Выберите видимость * Выберите, кто сможет просматривать этот репозиторий и вносить в него изменения **Публичный**

Добавить README Файлы README можно использовать для более подробного описания. [О файлах README](#) **Вкл**

Добавить .gitignore .gitignore сообщает git, какие файлы не нужно отслеживать. [Об игнорировании файлов](#) **Питон**

Добавить лицензию Лицензии определяют, как другие пользователи могут использовать ваш код. [Подробнее о лицензиях](#) **MIT License**

Рис. 1 - создание общедоступного репозитория с лицензией MIT, языком программирования Python.

```
PS C:\Users\user> git clone https://github.com/Vishhh123/Laborat_4.git
Cloning into 'Laborat_4'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
PS C:\Users\user> cd Laborat_4
```

Рис. 2 - клонирование репозитория на устройство, переход к репозиторию.

Ход работы:

```
[1]: import numpy as np
import pandas as pd
```

```
[2]: s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)

0 1
1 2
2 3
3 4
4 5
dtype: int64
```

```
[3]: s2 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
print(s2)

a 1
b 2
c 3
d 4
e 5
dtype: int64
```

```
[4]: ndarr = np.array([1, 2, 3, 4, 5])
type(ndarr)

numpy.ndarray
```

```
[5]: s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])
print(s3)

a 1
b 2
c 3
d 4
e 5
dtype: int64
```

```
[6]: a = 7
s5 = pd.Series(a, ['a', 'b', 'c'])
print(s5)

a 7
b 7
c 7
dtype: int64
```

```
[8]: s6 = pd.Series([1, 2, 3, 4, 5])
s6[2]
```

```
[8]: np.int64(3)
```

```
[9]: s6[s6 <= 3]
```

```
[9]: 0 1
1 2
2 3
dtype: int64
```

Рис. 3 – Импорт библиотек numpy и pandas для работы с массивами и структурой Series

```
[10]: s7 = pd.Series([10, 20, 30, 40, 50], ['a', 'b', 'c', 'd', 'e'])
      s6 + s7
      s6 * 3
```

```
[10]: 0 3
      1 6
      2 9
      3 12
      4 15
      dtype: int64
```

```
[11]: import pandas as pd

      s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

      print(s.iloc[0])
      print(s.iloc[2])
      print(s.iloc[-1])

      10
      30
      50
```

```
[12]: print(s.loc['a'])
      print(s.loc['c'])
      print(s.loc['e'])

      10
      30
      50
```

```
[13]: print(s.iloc[1:3])
      print(s.loc['b':'d'])

      b 20
      c 30
      dtype: int64
      b 20
      c 30
      d 40
      dtype: int64
```

```
[14]: import pandas as pd

      s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

      filtered_s = s[s > 10]

      print(filtered_s)

      b 25
      d 30
      e 15
      dtype: int64
```

```
[15]: print(s > 10)

      a False
      b True
      c False
      d True
      e True
      dtype: bool
```

Рис. 4 – Арифметическая операция умножения Series на число, доступ к элементам через `.iloc[]` и `.loc[]`, работа со срезами, логическая фильтрация данных и вывод булевого массива

```
[16]: filtered_s = s[(s >= 10) & (s <= 30)]
      print(filtered_s)

a 10
b 25
d 30
e 15
dtype: int64
```

```
[17]: filtered_s = s[s.index.isin(['b', 'd'])]
      print(filtered_s)

b 25
d 30
dtype: int64
```

```
[18]: s_with_nan = pd.Series([10, None, 8, 30, None], index=['a', 'b', 'c', 'd', 'e'])

      filtered_s = s_with_nan[s_with_nan.notnull()]
      print(filtered_s)

a 10,0
c 8,0
d 30,0
dtype: float64
```

```
[19]: import pandas as pd

      s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

      s.loc['b'] = 25
      print(s)

a 10
b 25
c 30
d 40
dtype: int64
```

```
[20]: s.iloc[1] = 50
      print(s)

a 10
b 50
c 30
d 40
dtype: int64
```

```
[21]: s[s > 30] += 10
      print(s)

a 10
b 60
c 30
d 50
dtype: int64
```

Рис. 5 — Фильтрация данных по диапазону значений с использованием логических операторов, фильтрация по индексам с помощью `.isin()`, фильтрация непустых значений через `.notnull()`, изменение элементов по индексу и по условию

```

..
[22]: s.loc[['a', 'c']] = [100, 200]
      print(s)

      a 100
      b 60
      c 200
      d 50
      dtype: int64

[23]: s = s.apply(lambda x: x * 2)
      print(s)

      a 200
      b 120
      c 400
      d 100
      dtype: int64

[24]: s_with_nan = pd.Series([10, None, 30, None], index=['a', 'b', 'c', 'd'])
      s_filled = s_with_nan.fillna(0)
      print(s_filled)

      a 10,0
      b 0,0
      c 30,0
      d 0,0
      dtype: float64

[26]: s = pd.Series([10, 20, 30, 40, 50, 60, 70], index=['a', 'b', 'c', 'd', 'e', 'f', 'g'])
      print(s.head(3))

      a 10
      b 20
      c 30
      dtype: int64

[27]: print(s.head())

      a 10
      b 20
      c 30
      d 40
      e 50
      dtype: int64

[28]: print(s.tail(3))

      e 50
      f 60
      g 70
      dtype: int64

[29]: print(s.tail())

      c 30
      d 40
      e 50
      f 60
      g 70
      dtype: int64

```

Рис. 6 – Изменение нескольких элементов по списку индексов, применение пользовательской функции к каждому элементу через `.apply()`, заполнение пропущенных значений с помощью `.fillna()`, а также просмотр первых и последних элементов Series с использованием методов `.head()` и `.tail()`

```
[30]: s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])  
  
print(s.index)  
  
Index(['a', 'b', 'c', 'd', 'e'], dtype='str')
```

```
[31]: print(s.index[0])  
print(s.index[-1])  
  
a  
e
```

```
[32]: print('b' in s.index)  
print('z' in s.index)  
  
Истина  
Ложь
```

```
[33]: print(s.values)  
  
[10 20 30 40 50]
```

```
[34]: print(s.values[0])  
print(s.values.mean())  
  
10  
30.0
```

```
[35]: s = pd.Series([10, 20, 30, 40, 50])  
  
print(s.dtype)  
  
int64
```

```
[36]: s1 = pd.Series([1.5, 2.3, 3.7])  
s2 = pd.Series(["apple", "banana", "cherry"])  
s3 = pd.Series([True, False, True])  
  
print(s1.dtype)  
print(s2.dtype)  
print(s3.dtype)  
  
float64  
str  
bool
```

```
[37]: s1_int = s1.astype(int)  
print(s1_int)  
print(s1_int.dtype)  
  
0 1  
1 2  
2 3  
dtype: int64  
int64
```

```
[38]: s4 = pd.Series([10, "apple", 3.14, True])  
print(s4.dtype)  
  
объект
```

Рис. 7 – Получение индексов и значений Series через атрибуты `.index` и `.values`, проверка принадлежности индекса, определение типа данных элементов с помощью `.dtype` и приведение типов через `.astype()`

```
[39]: import pandas as pd
import numpy as np

s = pd.Series([10, np.nan, 30, None, 50])

print(s.isnull())
```

```
0 False
1 True
2 False
3 True
4 False
dtype: bool
```

```
[40]: print(s.notnull())
```

```
0 True
1 False
2 True
3 False
4 True
dtype: bool
```

```
[41]: filtered_s = s[s.notnull()]
print(filtered_s)
```

```
0 10.0
2 30.0
4 50.0
dtype: float64
```

```
[42]: missing_values = s[s.isnull()]
print(missing_values)
```

```
1 NaN
3 NaN
dtype: float64
```

```
[43]: s = pd.Series([10, np.nan, 30, None, 50])

s_filled = s.fillna(0)
print(s_filled)
```

```
0 10.0
1 0.0
2 30.0
3 0.0
4 50.0
dtype: float64
```

```
[44]: s_filled = s.fillna(s.mean())
print(s_filled)
```

```
0 10.0
1 30.0
2 30.0
3 30.0
4 50.0
dtype: float64
```

Рис. 8 – Проверка наличия пропущенных значений в Series с помощью методов `.isnull()` и `.notnull()`, фильтрация пустых и непустых элементов, а также заполнение пропусков через `.fillna()` нулём и средним арифметическим значением


```
[45]: s_cleaned = s.dropna()
print(s_cleaned)

0 10,0
2 30,0
4 50,0
dtype: float64
```

```
[46]: import pandas as pd

s = pd.Series([10, 20, 30, 40, 50])

s_multiplied = s * 2
print(s_multiplied)

0 20
1 40
2 60
3 80
4 100
dtype: int64
```

```
[47]: s1 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s2 = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

s_sum = s1 + s2
print(s_sum)

a 11
b 22
c 33
d 44
e 55
dtype: int64
```

```
[48]: s3 = pd.Series([100, 200, 300], index=['a', 'b', 'f'])

s_result = s1 + s3
print(s_result)

a 101,0
b 202,0
c NaN
d NaN
e NaN
f NaN
dtype: float64
```

```
[49]: s_result = s1.add(s3, fill_value=0)
print(s_result)

a 101,0
b 202,0
c 3,0
d 4,0
e 5,0
f 300,0
dtype: float64
```

Рис. 9 – Удаление пропущенных значений через `.dropna()`, арифметические операции с Series (умножение на число, сложение с совпадающими и несовпадающими индексами), обработка NaN при сложении с помощью `fill_value`

```
[1]: import pandas as pd

s = pd.Series([10, 20, 30, 40, 50])

print(s.sum())

150

[2]: s_with_nan = pd.Series([10, 20, None, 40, 50])

print(s_with_nan.sum())

120.0

[3]: print(s.mean())

30.0

[4]: print(s_with_nan.mean())

30.0

[5]: print(s.min())
print(s.max())

10
50

[6]: print(s.describe())

количество 5.000000
среднее значение 30.000000
стандартное отклонение 15.811388
мин 10.000000
25% 20.000000
50% 30.000000
75% 40.000000
макс 50.000000
тип данных: float64

[7]: s_text = pd.Series(["apple", "banana", "apple", "cherry", "banana"])
print(s_text.describe())

количество 5
уникальных 3
верхнее яблоко
частота 2
тип данных: объект

[9]: import pandas as pd
import numpy as np

s = pd.Series([1, 2, 3, 4, 5])

s_log = np.log(s)
print(s_log)

0 0,000000
1 0,693147
2 1,098612
3 1,386294
4 1,609438
dtype: float64
```

Рис. 10 – Применение агрегирующих функций `.sum()`, `.mean()`, `.min()`, `.max()`, `.describe()` для вычисления статистических показателей числовых и строковых данных, а также применение математической функции `np.log()` к элементам `Series`

```
[10]: s_exp = np.exp(s)
      print(s_exp)
```

```
0 2,718282
1 7,389056
2 20,085537
3 54,598150
4 148,413159
dtype: float64
```

```
[11]: print(np.sin(s))
      print(np.cos(s))
      print(np.abs(s))
```

```
0 0,841471
1 0,909297
2 0,141120
3 -0,756802
4 -0,958924
dtype: float64
0 0,540302
1 -0,416147
2 -0,989992
3 -0,653644
4 0,283662
dtype: float64
0 1
1 2
2 3
3 4
4 5
dtype: int64
```

```
[12]: s_with_nan = pd.Series([1, np.nan, 4, 9])
      print(np.sqrt(s_with_nan))
```

```
0 1.0
1 NaN
2 2.0
3 3.0
dtype: float64
```

```
[13]: import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
[14]: s.index = ['x', 'y', 'z', 'w']
      print(s)
```

```
x 10
y 20
z 30
w 40
dtype: int64
```

Рис. 11 — Применение математических функций NumPy к элементам Series: экспонента, синус, косинус, модуль и квадратный корень, а также изменение индексов Series путём присвоения нового списка меток

```

[15]: s_reset = s.reset_index()
      print(s_reset)

      индекс 0
      0 x 10
      1 y 20
      2 z 30
      3 w 40

[16]: s_reset = s.reset_index(drop=True)
      print(s_reset)

      0 10
      1 20
      2 30
      3 40
      dtype: int64

[17]: import pandas as pd

      s_unique = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
      print(s_unique.index.is_unique)

      Истинный

[18]: s_duplicate = pd.Series([10, 20, 30, 40], index=['a', 'b', 'a', 'c'])
      print(s_duplicate.index.is_unique)

      Ложь

[19]: print(s_duplicate.loc['a'])

      a 10
      a 30
      dtype: int64

[20]: s_reset = s_duplicate.reset_index(drop=True)
      print(s_reset)

      0 10
      1 20
      2 30
      3 40
      dtype: int64

[21]: s_duplicate.index = [f"{i}_{n}" for n, i in enumerate(s_duplicate.index)]
      print(s_duplicate)

      a_0 10
      b_1 20
      a_2 30
      c_3 40
      dtype: int64

[22]: s = pd.Series([10, 20, 30, 40], index=['d', 'b', 'a', 'c'])
      s_sorted = s.sort_index()
      print(s_sorted)

      a 30
      b 20
      c 40
      d 10
      dtype: int64

```

Рис. 12 – Сброс индексов с помощью `.reset_index()` с сохранением и удалением старого индекса, проверка уникальности индексов через `.is_unique`, доступ к элементам при повторяющихся индексах, преобразование индексов в уникальные с добавлением суффиксов, а также сортировка Series по индексу через `.sort_index()`

```
[23]: s_sorted_desc = s.sort_index(ascending=False)
      print(s_sorted_desc)

d 10
c 40
b 20
a 30
dtype: int64

[24]: s_sorted_values = s.sort_values()
      print(s_sorted_values)

d 10
b 20
a 30
c 40
dtype: int64

[25]: s_sorted_values_desc = s.sort_values(ascending=False)
      print(s_sorted_values_desc)

c 40
a 30
b 20
d 10
dtype: int64

[26]: s_nan = pd.Series([10, None, 30, None, 20], index=['a', 'b', 'c', 'd', 'e'])
      print(s_nan.sort_values())

a 10,0
e 20,0
c 30,0
b NaN
d NaN
dtype: float64

[27]: print(s_nan.sort_values(na_position='first'))

b NaN
d NaN
a 10,0
e 20,0
c 30,0
dtype: float64

[28]: import pandas as pd

dates = pd.date_range(start='2024-03-01', periods=5, freq='D')
s = pd.Series([100, 105, 102, 98, 110], index=dates)
print(s)

2024-03-01 100
2024-03-02 105
2024-03-03 102
2024-03-04 98
2024-03-05 110
Частота: D, тип данных: int64
```

Рис. 13 – Сортировка Series по индексу и по значениям в порядке возрастания и убывания, обработка пропущенных значений при сортировке с параметром `na_position`, а также создание временного ряда с использованием `pd.date_range()`

```
[29]: print(s['2024-03-03'])
      print(s['2024-03-02':'2024-03-04'])

102
2024-03-02 105
2024-03-03 102
2024-03-04 98
Частота: D, тип данных: int64

[34]: dates = pd.date_range(start='2024-03-01', periods=5, freq='h')
      s = pd.Series([10, 15, 12, 8, 20], index=dates)
      print(s)

2024-03-01 00:00:00 10
2024-03-01 01:00:00 15
2024-03-01 02:00:00 12
2024-03-01 03:00:00 8
2024-03-01 04:00:00 20
Частота: ч, тип данных: int64

[31]: s = pd.Series([100, 200, 150], index=['2024-03-01', '2024-03-02', '2024-03-03'])
      s.index = pd.to_datetime(s.index)
      print(s)

2024-03-01 100
2024-03-02 200
2024-03-03 150
dtype: int64

[32]: print(s.resample('D').mean())
      print(s[s.index.year == 2024])

2024-03-01 100,0
2024-03-02 200,0
2024-03-03 150,0
Частота: D, тип данных: float64
2024-03-01 100
2024-03-02 200
3 марта 2024 года, 150
dtype: int64

[33]: print(s.shift(1))

2024-03-01 NaN
2024-03-02 100,0
2024-03-03 200,0
dtype: float64

[35]: import pandas as pd

      s = pd.Series([10, 20, 30, 40, 50, 60, 70], index=pd.date_range("2024-03-01", periods=7, freq="D"))
      s_ma = s.rolling(window=3).mean()
      print(s_ma)

2024-03-01 NaN
2024-03-02 NaN
2024-03-03 20.0
2024-03-04 30.0
2024-03-05 40.0
2024-03-06 50.0
2024-03-07 60.0
Частота: D, тип данных: float64
```

Рис. 14 – Доступ к элементам временного ряда по конкретной дате и диапазону дат, создание временного ряда с часовой частотой, преобразование строковых дат в DatetimeIndex, ресемплинг данных по дням, фильтрация по году, сдвиг временного ряда с помощью `.shift()`, а также вычисление скользящего среднего с использованием `.rolling().mean()`

```
[36]: s_ma5 = s.rolling(window=5).mean()
print(s_ma5)
```

```
2024-03-01 NaN
2024-03-02 NaN
2024-03-03 NaN
2024-03-04 NaN
2024-03-05 30.0
2024-03-06 40.0
2024-03-07 50.0
Частота: D, тип данных: float64
```

```
[37]: s_ma2 = s.rolling(window=3, min_periods=1).mean()
print(s_ma2)
```

```
2024-03-01 10.0
2024-03-02 15.0
2024-03-03 20.0
2024-03-04 30.0
2024-03-05 40.0
2024-03-06 50.0
2024-03-07 60.0
Частота: D, тип данных: float64
```

```
[38]: import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 4))
plt.plot(s, label="Исходные данные", marker="o")
plt.plot(s.rolling(window=3).mean(), label="Скользящее среднее (3)", linestyle="--", color="red")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.title("Скользящее среднее в Series")
plt.legend()
plt.grid()
plt.show()
```

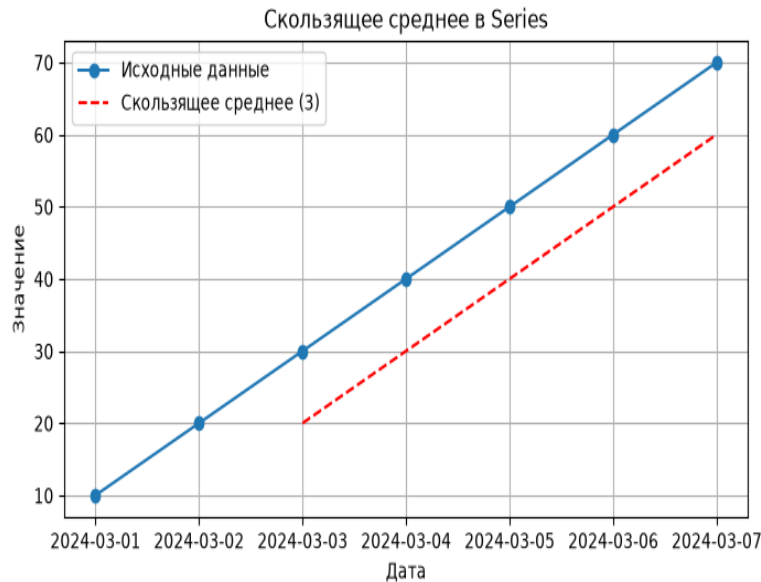


Рис. 15 – Вычисление скользящего среднего с различными размерами окна и минимальным количеством значений (`min_periods`), а также визуализация исходных данных и сглаженного тренда с помощью библиотеки `matplotlib`

```
[39]: s = pd.Series([100, 110, 120, 90, 150])
print(s.pct_change())

0 NaN
1 0.100000
2 0.090909
3 -0.250000
4 0.666667
dtype: float64
```

```
[40]: print(s.pct_change() * 100)

0 NaN
1 10.000000
2 9.090909
3 -25.000000
4 66.666667
dtype: float64
```

```
[41]: print(s.pct_change(periods=2))

0 NaN
1 NaN
2 0.200000
3 -0.181818
4 0.250000
dtype: float64
```

```
[42]: import numpy as np

dates = pd.date_range(start="2024-01-01", periods=30, freq="D")
prices = pd.Series(np.random.randint(90, 110, size=30), index=dates)
returns = prices.pct_change() * 100
```

```
[43]: plt.figure(figsize=(10, 5))
plt.plot(prices, label="Цена акции", marker="o")
plt.xlabel("Цена")
plt.ylabel("Дата")
plt.title("График цены акции")
```

```
[43]: Text(0.5, 1.0, «График цены акции»)
```

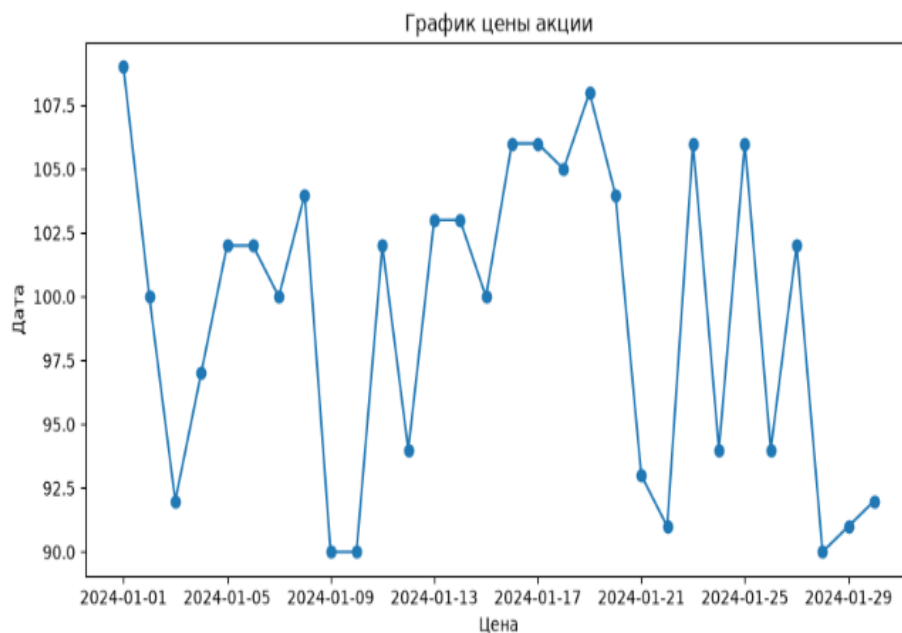
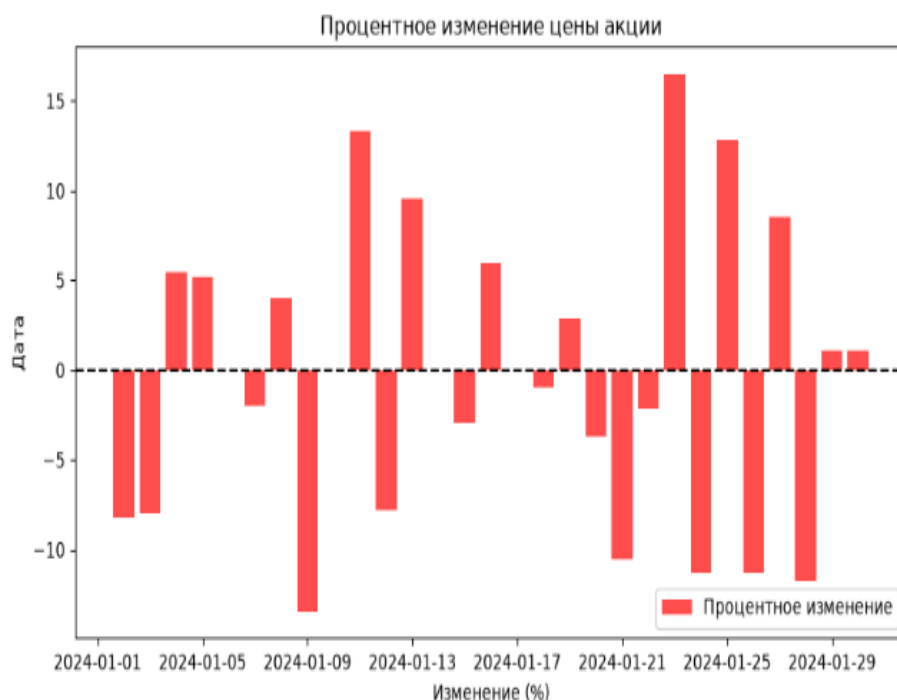


Рис. 16 – Вычисление процентного изменения с помощью `.pct_change()` с разными периодами, визуализация цен акций и процентного прироста на графиках с использованием библиотеки `matplotlib`


```
[44]: plt.figure(figsize=(10, 5))
plt.bar(returns.index, returns, color="red", alpha=0.7, label="Процентное изменение")
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Изменение (%)")
plt.ylabel("Дата")
plt.title("Процентное изменение цены акции")
plt.legend()
plt.show()
```



```
[76]: import pandas as pd
import os

def ensure_csv(filename, data):
    if not os.path.exists(filename):
        df = pd.DataFrame(data)
        df.to_csv(filename, index=False)
        print(f"{filename}")
    else:
        print(f"{filename}")
```

```
[68]: df = pd.read_csv("data.csv")
s = df["Цена"]
print(s)
```

```
0 100
1 110
2 105
3 120
4 115
Название: Цена, тип данных: int64
```

Рис. 17 – Визуализация процентного изменения цены акции с помощью столбчатой диаграммы, а также создание функции для проверки наличия CSV-файла и его автоматического создания, чтение данных из CSV-файла и создание Series на основе столбца "Цена"

```
[69]: df = pd.read_csv("data.csv", header=None, names=["Дата", "Цена"])
      s = df["Цена"]

[83]: import pandas as pd

      data = {
          "Дата": ["2024-03-01", "2024-03-02", "2024-03-03", "2024-03-04", "2024-03-05"],
          "Объём продаж": [100, 110, 105, 120, 115]
      }

      df = pd.DataFrame(data)
      df.to_excel("data.xlsx", sheet_name="Лист1", index=False)
      print("Файл data.xlsx создан")

      df = pd.read_excel("data.xlsx", sheet_name="Лист1")
      s = df["Объём продаж"]
      print(s)

      0 100
      1 110
      2 105
      3 120
      4 115
      Название: Объём продаж, тип данных: int64

[84]: df = pd.read_excel("data.xlsx", sheet_name="Лист1")
      s = df["Объём продаж"]
      print(s)

      0 100
      1 110
      2 105
      3 120
      4 115
      Название: Объём продаж, тип данных: int64

[85]: s = pd.Series(df["Объём продаж"].values, index=df["Дата"])
      print(s)

      Дата
      2024-03-01 100
      2024-03-02 110
      2024-03-03 105
      2024-03-04 120
      2024-03-05 115
      dtype: int64

[86]: s = pd.read_csv("data.csv", index_col="Дата")["Цена"]
      print(s)

      Дата
      2024-03-01 100
      2024-03-02 110
      2024-03-03 105
      2024-03-04 120
      2024-03-05 115
      Название: Цена, тип данных: int64

[87]: s = pd.read_excel("data.xlsx", index_col="Дата")["Объём продаж"]
      print(s)

      Дата
      2024-03-01 100
      2024-03-02 110
      2024-03-03 105
      2024-03-04 120
      2024-03-05 115
      Название: Объём продаж, тип данных: int64

[ ]:
```

Рис. 18 – Чтение данных из CSV-файла без заголовков с указанием имён столбцов, создание и сохранение данных в Excel-файл с последующим чтением, создание Series с указанием индекса из столбца "Дата", а также установка столбца "Дата" в качестве индекса при чтении CSV и Excel-файлов

Задания:

1. Создание Series из списка

Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e']. Выведите его на экран и определите его тип данных.

```
# 1. Создание Series из списка
import pandas as pd

s = pd.Series([5, 15, 25, 35, 45], index=['a', 'b', 'c', 'd', 'e'])
print(s)
print(s.dtype)

a 5
b 15
c 25
d 35
e 45
dtype: int64
int64
```

Рис. 19 – создание Series из списка чисел с заданными строковыми индексами, вывод содержимого и определение типа данных элементов

2. Получение элемента Series

Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60]. Используйте .loc[] для получения элемента с индексом 'C' и .iloc[] для получения третьего элемента.

```
: # 2. Получение элемента Series
import pandas as pd

s = pd.Series([12, 24, 36, 48, 60], index=['A', 'B', 'C', 'D', 'E'])
print(s.loc['C'])
print(s.iloc[2])

36
36
```

Рис. 20 - доступ к элементам Series с использованием методов .loc[] по метке и .iloc[] по позиции

3. Фильтрация данных с помощью логической индексации

Создайте `Series` из массива NumPy `np.array([4, 9, 16, 25, 36, 49, 64])`. Выберите только те элементы, которые больше 20, и выведите результат.

```
# 3. Фильтрация данных с помощью логической индексации
import pandas as pd
import numpy as np

s = pd.Series(np.array([4, 9, 16, 25, 36, 49, 64]))
print(s[s > 20])
```

```
3 25
4 36
5 49
6 64
dtype: int64
```

Рис. 21 - Создание `Series` из массива NumPy и фильтрация элементов, значения которых больше 20, с помощью логической индексации

4. Просмотр первых и последних элементов

Создайте `Series`, содержащий 50 случайных целых чисел от 1 до 100 (используйте `np.random.randint`). Выведите первые 7 и последние 5 элементов с помощью `.head()` и `.tail()`.

```
# 4. Просмотр первых и последних элементов
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(1, 100, 50))
print(s.head(7))
print(s.tail(5))
```

```
0 59
1 64
2 64
3 86
4 79
5 86
6 11
dtype: int32
45 30
46 60
47 86
48 32
49 51
dtype: int32
```

Рис. 22 - Создание Series из 50 случайных чисел и просмотр первых 7 элементов с помощью `.head()` и последних 5 элементов с помощью `.tail()`

5. Определение типа данных Series

Создайте Series из списка `['cat', 'dog', 'rabbit', 'parrot', 'fish']`. Определите тип данных с помощью `.dtype`, затем преобразуйте его в `category` с помощью `.astype()`.

```
# 5. Определение типа данных Series
import pandas as pd

s = pd.Series(['cat', 'dog', 'rabbit', 'parrot', 'fish'])
print(s.dtype)
s = s.astype('category')
print(s.dtype)

str
категория
```

Рис. 23 - Определение типа данных элементов Series с помощью атрибута `.dtype` и преобразование типа в категориальный с использованием `.astype('category')`

6. Проверка пропущенных значений

Создайте `Series` с данными `[1.2, np.nan, 3.4, np.nan, 5.6, 6.8]`. Напишите код, который проверяет, есть ли в `Series` пропущенные значения (`NaN`), и выведите индексы таких элементов.

```
# 6. Проверка пропущенных значений
import pandas as pd
import numpy as np

s = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
print(s.isnull())
print(s[s.isnull()].index)

0 False
1 True
2 False
3 True
4 False
5 False
dtype: bool
RangeIndex(start=1, stop=5, step=2)
```

Рис. 24 - Проверка наличия пропущенных значений в `Series` с помощью метода `.isnull()` и вывод индексов элементов, содержащих `NaN`

7. Заполнение пропущенных значений

Используйте `Series` из предыдущего задания и замените все `NaN` на среднее значение всех непустых элементов. Выведите результат.

```
# 7. Заполнение пропущенных значений
import pandas as pd
import numpy as np

s = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
s_filled = s.fillna(s.mean())
print(s_filled)
```

```
0 1,20
1 4,25
2 3,40
3 4,25
4 5,60
5 6,80
dtype: float64
```

Рис. 25 - Заполнение пропущенных значений средним арифметическим всех непустых элементов с использованием метода `.fillna()`

8. Арифметические операции с Series

Создайте два Series :

- `s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])`
- `s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])`

Выполните сложение `s1 + s2` . Объясните, почему в результате появляются `NaN` , и замените их на `0` .

8. Арифметические операции с Series

```
import pandas as pd
```

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
```

```
result = s1 + s2
```

```
print(result)
```

```
result_filled = s1.add(s2, fill_value=0)
```

```
print(result_filled)
```

```
a NaN
```

```
b 25,0
```

```
c 45,0
```

```
d 65,0
```

```
e NaN
```

```
dtype: float64
```

```
a 10,0
```

```
b 25,0
```

```
c 45,0
```

```
d 65,0
```

```
e 35,0
```

```
dtype: float64
```

Рис. 26 - Сложение двух Series с несовпадающими индексами, появление NaN из-за отсутствия общих меток и замена пропущенных значений на 0 с помощью параметра fill_value в методе .add()

9. Применение функции к Series

Создайте Series из чисел [2, 4, 6, 8, 10]. Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью .apply(np.sqrt).


```

: # 9. Применение функции к Series
import pandas as pd
import numpy as np

s = pd.Series([2, 4, 6, 8, 10])
s_sqrt = s.apply(np.sqrt)
print(s_sqrt)

0 1,414214
1 2,000000
2 2,449490
3 2,828427
4 3,162278
dtype: float64

```

Рис. 27 - Применение функции вычисления квадратного корня к каждому элементу Series с использованием метода `.apply()` и библиотечной функции `np.sqrt`

10. Основные статистические методы

Создайте Series из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

```

# 10. Основные статистические методы
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(50, 150, 20))
print(f"Сумма: {s.sum()}")
print(f"Среднее: {s.mean()}")
print(f"Минимум: {s.min()}")
print(f"Максимум: {s.max()}")
print(f"Стандартное отклонение: {s.std()}")

Сумма: 1982
Среднее: 99,1
Минимум: 50
Максимум: 147
Стандартное отклонение: 31,066821022129904

```

Рис. 28 - Применение агрегирующих статистических методов к Series: вычисление суммы, среднего, минимального и максимального значения, а также стандартного отклонения

11. Работа с временными рядами

Создайте `Series`, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`), а значениями – случайные числа от 10 до 100. Выберите данные за 5–8 марта.

```
# 11. Работа с временными рядами
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(10, 100, 10), index=pd.date_range(start='2024-03-01', periods=10, freq='D'))
print(s['2024-03-05':'2024-03-08'])

2024-03-05 61
2024-03-06 36
2024-03-07 18
2024-03-08 27
Частота: D, тип данных: int32
```

Рис. 30 - Создание временного ряда с индексами-датами и выборка данных за определённый диапазон дат (с 5 по 8 марта 2024 года)

12. Проверка уникальности индексов

Создайте `Series` с индексами `['A', 'B', 'A', 'C', 'D', 'B']` и значениями `[10, 20, 30, 40, 50, 60]`. Проверьте, являются ли индексы уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

```
# 12. Проверка уникальности индексов
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C', 'D', 'B'])
print(s.index.is_unique)
s_grouped = s.groupby(s.index).sum()
print(s_grouped)

False
A 40
B 80
C 40
D 50
dtype: int64
```

Рис. 31 - Проверка уникальности индексов с помощью атрибута `.is_unique` и группировка повторяющихся индексов с суммированием их значений через `.groupby().sum()`

13. Преобразование строковых дат в DatetimeIndex

Создайте `Series`, где индексами будут строки `['2024-03-10', '2024-03-11', '2024-03-12']`, а значениями `[100, 200, 300]`. Преобразуйте индексы в `DatetimeIndex` и выведите тип данных индекса.

```
# 13. Преобразование строковых дат в DatetimeIndex
import pandas as pd

s = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '2024-03-12'])
s.index = pd.to_datetime(s.index)
print(s.index.dtype)

datetime64[us]
```

Рис. 32 - Преобразование строковых индексов с датами в формат `DatetimeIndex` с помощью `pd.to_datetime()` и вывод типа данных индекса

14. Чтение данных из CSV-файла

Создайте CSV-файл `data.csv` со следующими данными:



```
1  Дата, Цена
2  2024-03-01, 100
3  2024-03-02, 110
4  2024-03-03, 105
5  2024-03-04, 120
6  2024-03-05, 115
```

Прочитайте файл и создайте `Series`, используя "Дата" в качестве индекса.

14. Чтение данных из CSV-файла

```
import pandas as pd
```

```
data = {  
    "Дата": ["2024-03-01", "2024-03-02", "2024-03-03", "2024-03-04", "2024-03-05"],  
    "Цена": [100, 110, 105, 120, 115]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("data.csv", index=False)
```

```
s = pd.read_csv("data.csv", index_col="Дата")["Цена"]
```

```
print(s)
```

Дата

2024-03-01 100

2024-03-02 110

2024-03-03 105

2024-03-04 120

2024-03-05 115

Название: Цена, тип данных: int64

Рис. 33 - Создание CSV-файла с данными и чтение его с установкой столбца "Дата" в качестве индекса Series с помощью параметра `index_col`

15. Построение графика на основе Series

Создайте `Series`, где индексами будут даты с 1 по 30 марта 2024 года, а значениями – случайные числа от 50 до 150. Постройте график значений с помощью `matplotlib`. Добавьте заголовок, подписи осей и сетку.

```
# 15. Построение графика на основе Series
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

s = pd.Series(np.random.randint(50, 150, 30), index=pd.date_range(start='2024-03-01', periods=30, freq='D'))

plt.figure(figsize=(10, 6))
plt.plot(s, marker='o')
plt.title('График значений Series')
plt.xlabel('Дата')
plt.ylabel('Значение')
plt.grid()
plt.show()
```

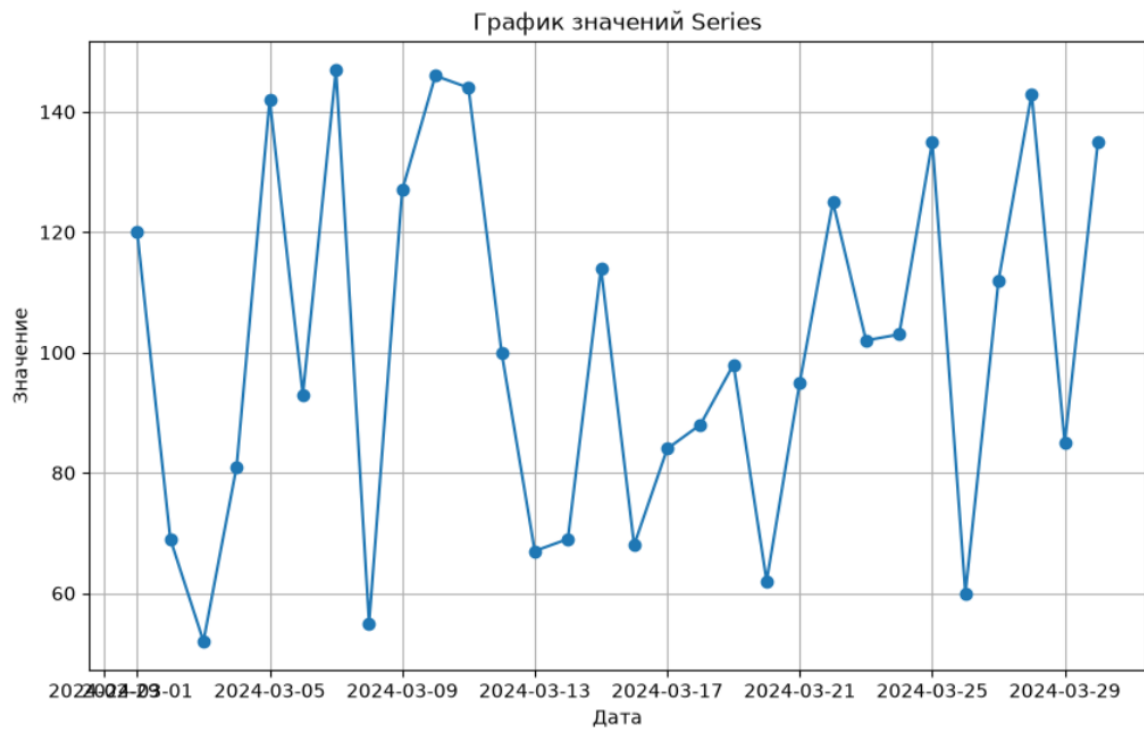


Рис. 34 - Построение графика временного ряда с использованием библиотеки matplotlib: визуализация случайных значений за март 2024 года с заголовком, подписями осей и сеткой

25. Оценка спроса на товар в магазине

Создайте `Series`, где индексами будут дни с 1 по 20 апреля 2024 года, а значениями – количество проданных единиц товара (случайные значения от 20 до 100).

Рассчитайте процентное изменение продаж (`pct_change()` * 100) и найдите дни с наибольшими изменениями.

```

: # 25. Оценка спроса на товар в магазине
import pandas as pd
import numpy as np

s = pd.Series(np.random.randint(20, 100, 20), index=pd.date_range(start='2024-04-01', periods=20, freq='D'))

pct_change = s.pct_change() * 100
print(pct_change)

print(f"Максимальный рост: {pct_change.max():.2f}%")
print(f"Максимальное падение: {pct_change.min():.2f}%")

```

2024-04-01 NaN
 2024-04-02 -21,794872
 2024-04-03 1,639344
 2024-04-04 24,193548
 2024-04-05 -20,779221
 2024-04-06 -44,262295
 2024-04-07 -38,235294
 2024-04-08 176,190476
 2024-04-09 25,862069
 2024-04-10 -1,369863
 2024-04-11 2,777778
 2024-04-12 27,027027
 2024-04-13 4,255319
 2024-04-14 -44,897959
 2024-04-15 42,592593
 2024-04-16 27,272727
 2024-04-17 -4,081633
 2024-04-18 -68,085106
 2024-04-19 73,333333
 2024-04-20 67,307692
 Частота: D, тип данных: float64
 Максимальный рост: 176,19%
 Максимальное падение: -68,09%

Рис. 35 - Оценка спроса на товар: расчёт процентного изменения продаж с помощью `.pct_change()` и определение дней с максимальным ростом и падением

Ответы на контрольные вопросы:

1. Что такое `pandas.Series` и чем она отличается от списка в Python?
`Series` — это одномерная маркированная структура данных с индексами (метками). От списка отличается наличием индексов, которые могут быть не только числовыми, но и строковыми, что позволяет обращаться к элементам по меткам.
2. Какие типы данных можно использовать для создания `Series`?
 Списки Python, массивы NumPy (`ndarray`), словари (`dict`) и скалярные величины.
3. Как задать индексы при создании `Series`?
 Вторым параметром в конструкторе: `pd.Series([1,2,3], ['a','b','c'])`
4. Каким образом можно обратиться к элементу `Series` по его индексу?
 По метке: `s['a']` или `s.loc['a']`. По позиции: `s.iloc[0]`.
5. В чём разница между `.iloc[]` и `.loc[]`?
`.iloc[]` — доступ по позиции (числовому индексу), срез НЕ включает последний.
`.loc[]` — доступ по метке, срез включает последний.
6. Как использовать логическую индексацию в `Series`?

Создаётся булев массив: `s[s > 10]` — вернёт элементы, где условие True.

7. Какие методы для просмотра первых и последних элементов?

`.head(n)` — первые `n` элементов. `.tail(n)` — последние `n` элементов.

8. Как проверить тип данных элементов Series?

Атрибут `.dtype: s.dtype`

9. Как изменить тип данных Series?

`.astype(): s.astype('category')`

10. Как проверить наличие пропущенных значений?

`.isnull()` — возвращает True для NaN. `.notnull()` — наоборот.

11. Какие методы для заполнения пропущенных значений?

`.fillna(value)` — заменяет NaN на указанное значение.

12. Чем отличается `.fillna()` от `.dropna()`?

`.fillna()` — заменяет NaN на значение.

`.dropna()` — удаляет строки с NaN.

13. Какие математические операции можно выполнять с Series?

Сложение, вычитание, умножение, деление на число или другую Series (поэлементно).

14. В чём преимущество векторизованных операций?

Работают быстрее циклов `for`, выполняются сразу над всеми элементами (в коде на C).

15. Как применить пользовательскую функцию к каждому элементу?

`.apply(func): s.apply(lambda x: x**2)`

16. Какие агрегирующие функции доступны?

`.sum()`, `.mean()`, `.min()`, `.max()`, `.std()`, `.describe()`

17. Как узнать минимум, максимум, среднее и стандартное отклонение?

`.min()`, `.max()`, `.mean()`, `.std()`

18. Как сортировать Series по значениям и индексам?

По значениям: `.sort_values()`. По индексам: `.sort_index()`.

19. Как проверить уникальность индексов?

`s.index.is_unique` — возвращает True/False.

20. Как сбросить индексы и сделать числовыми?

`.reset_index(drop=True)`

21. Как задать новый индекс?

Присвоить: `s.index = ['x','y','z']`

22. Как работать с временными рядами?

Создать индекс с датами через `pd.date_range()` и использовать его как индекс Series.

23. Как преобразовать строковые даты в DatetimeIndex?

`s.index = pd.to_datetime(s.index)`

24. Как выбрать данные за диапазон дат?

`['2024-03-02':'2024-03-04']` — как срез.

25. Как загрузить данные из CSV в Series?

`df = pd.read_csv("file.csv")`, затем взять столбец: `s = df["Цена"]`

26. Как установить столбец CSV в качестве индекса?

`pd.read_csv("file.csv", index_col="Дата")`

27. Для чего используется `.rolling().mean()`?

Вычисляет скользящее среднее — сглаживает данные, показывая тренд.

28. Как работает `.pct_change()`? Какие задачи решает?

Вычисляет процентное изменение между текущим и предыдущим значением: $(\text{текущее} - \text{предыдущее}) / \text{предыдущее} * 100\%$. Используется в финансовом анализе.

29. Когда полезно использовать `.rolling()` и `.pct_change()`?

`.rolling()` — для сглаживания трендов, поиска аномалий.

`.pct_change()` — для анализа прироста/падения (цены акций, продажи).

30. Почему NaN появляются и как с ними работать?

Появляются при операциях с несовпадающими индексами.

Работа: `.fillna()`, `.dropna()`, `.isnull()`.

Вывод: В ходе лабораторной работы изучена структура `pandas.Series`: создание из списков, массивов NumPy и словарей, работа с индексами, доступ к элементам через `.loc[]` и `.iloc[]`. Освоены фильтрация данных, обработка пропущенных значений (`.isnull()`, `.fillna()`, `.dropna()`), векторизованные операции, агрегирующие функции (`.sum()`, `.mean()`, `.describe()`), сортировка и работа с временными рядами (`pd.date_range()`, `DatetimeIndex`). Рассмотрены скользящее среднее (`.rolling().mean()`) и процентный прирост (`.pct_change()`)